

# Benchmarking and Adaptive Optimization of Filtered Vector Search on Proximity Graphs

Concept proposal for supervisor discussion | A diagnostic-driven, CPU-oriented research plan

|                  |                                                      |                |                                                     |
|------------------|------------------------------------------------------|----------------|-----------------------------------------------------|
| Investigator     | Shafikul Islam Marwan                                | Project Type   | Undergraduate Thesis / Independent Systems Research |
| Domain           | Database Systems, Vector Databases, Graph Algorithms | Infrastructure | Commodity multi-core CPU; no GPU dependency         |
| Potential Venues | ACM CIKM, ACM SAC, SISAP, IEEE BigData               | Study Period   | 6 months initial investigation                      |

## 1. Executive Summary

Filtered Approximate Nearest Neighbor Search (FANNS) retrieves the nearest vectors to a query subject to structured attribute constraints. This setting is important in vector databases, neural search, recommendation, and Retrieval-Augmented Generation (RAG) systems. Proximity-graph indexes such as HNSW and Vamana/DiskANN are strong foundations for vector search, but filtering can make the set of valid nodes sparse relative to the underlying graph and can change the cost and accuracy of graph traversal. This project does not assume a novel algorithm in advance. It proposes a diagnostic-driven study: first reproduce and profile representative filtered-search methods on commodity CPUs, then identify a reproducible performance bottleneck, and finally develop a lightweight adaptive traversal technique targeted at that measured bottleneck. The main goal is to understand where search effort is unnecessarily spent and whether it can be reduced without rebuilding the index.

## 2. Problem and Motivation

|                                          |                                                  |                                                   |                                                                |
|------------------------------------------|--------------------------------------------------|---------------------------------------------------|----------------------------------------------------------------|
| Vector query<br>Find top-k similar items | Metadata filter<br>brand = Nike AND price < \$50 | Graph traversal<br>Explore HNSW / proximity graph | Challenge<br>Valid nodes may be sparse; search effort can rise |
|------------------------------------------|--------------------------------------------------|---------------------------------------------------|----------------------------------------------------------------|

- Pre-filtering can reduce the candidate set before graph search, but may lose the benefit of the original proximity index when the filtered subset is small or poorly represented.
- Post-filtering can waste search effort because the nearest unfiltered neighbors may not satisfy the predicate, especially under selective filters.
- In-graph filtered traversal preserves graph navigation but may inspect many predicate-failing nodes or explore extra paths when valid nodes are sparse.

The research question is therefore not simply whether filtering can be accelerated, but: under which filter selectivities, query-filter correlations, and graph-search conditions does existing CPU filtered search spend excessive work, and can a small runtime controller reduce that work without materially reducing recall?

## 3. Research Position and Hypothesis

Recent work already covers filter-aware graph indexing, predicate-aware traversal, local-selectivity adaptation, adaptive fallback, attribute-aware graphs, and online beam adaptation. Therefore, the proposed contribution will be defined only after empirical comparison rather than by claiming that adaptive traversal itself is new.

Working hypothesis: A lightweight query-time controller, informed by early traversal signals and constrained by a recall target, can reduce redundant distance computations and tail latency on CPU-based filtered graph search for at least one reproducible workload regime.

## 4. Proposed 3-Phase Methodology

|                                  |                                                                                                                                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phase 1<br>Benchmark & reproduce | Months 1-2<br>Build and validate representative HNSW / filtered-search baselines; sweep selectivity and query-filter correlation; establish reproducible recall and latency curves.           |
| Phase 2<br>Profile & diagnose    | Month 3<br>Use Linux perf and algorithmic counters to separate predicate evaluation, distance computation, graph bookkeeping, and cache/stall costs; identify one or more bottleneck regimes. |
| Phase 3<br>Adaptive refinement   | Months 4-6<br>Design and implement one evidence-driven runtime heuristic, then run ablations and compare against the same tuned baselines.                                                    |

## 5. Evaluation Plan

- Workloads: filtered-ANN benchmark suites and public filtered-search workloads, including MoReVec-related and Qdrant-style workloads where compatible.
- Stress factors: selectivity from approximately 0.01% to 50%; random, positively correlated, and negatively correlated query-filter distributions; multiple k values.
- Metrics: Recall@K, p50/p95/p99 latency, QPS, distance computations, nodes visited, and index/build overhead.
- Validation: fixed hardware, repeated runs, matched parameter budgets, ablation studies, and exact-search ground truth where feasible.

## 6. Feasibility and Deliverables

- CPU-oriented implementation in C++ with Python-based experiment automation. No dedicated GPU cluster is required.
  - Deliverables: reproducible benchmark harness, failure-mode analysis, one targeted adaptive traversal prototype, experimental report, and manuscript-ready draft.
- The project deliberately avoids promising a fixed percentage improvement or guaranteed publication outcome before the baseline study. The research contribution will be determined by the evidence produced during Phases 1 and 2.

# Literature Context and Research Positioning

The table below is intended for discussion with the supervisor. It summarizes representative recent work and states how this project plans to position its own investigation.

| Method / Work           | Year / Venue  | Core idea                                                                                           | How this project uses it                                                                                |
|-------------------------|---------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Filtered-DiskANN        | 2023 / WWW    | Filter-aware Vamana-style graph indexing for filtered ANNS.                                         | Reference for index-time filtering and graph construction baselines.                                    |
| ACORN                   | 2024 / SIGMOD | Predicate-aware traversal over HNSW with predicate subgraph exploration.                            | Primary query-time filtered traversal baseline.                                                         |
| NaviX                   | 2025 / PVLDB  | Adaptive HNSW search using local selectivity under varying query-filter correlation.                | Important baseline when testing whether local runtime signals add value.                                |
| FANNS Unified Benchmark | 2025 / arXiv  | Systematic evaluation framework and analysis of selectivity, query difficulty, and dataset effects. | Guides fair experimental design and parameter tuning.                                                   |
| JAG                     | 2026 / arXiv  | Joint attribute and vector graphs with attribute/filter distances for navigation.                   | Recent alternative to compare against when studying graph structure versus runtime adaptation.          |
| RACORN-1                | 2026 / arXiv  | Adaptive fallback and bridge-style routing for low-selectivity filtered search.                     | Recent low-selectivity baseline; helps define what the proposed method must improve upon or complement. |
| BOA                     | 2026 / arXiv  | Online beamwidth adaptation for filtered ANNS on a GPU.                                             | Motivates a CPU-focused comparison of adaptive search effort, without assuming CPU novelty a priori.    |

## 7. Research Positioning Statement

The project is deliberately framed as benchmark -> diagnose -> refine. The literature already demonstrates that filtered vector search can be improved through index construction, predicate-aware traversal, local-selectivity adaptation, fallback strategies, attribute-aware graphs, and online beam adaptation. The proposed work will therefore claim novelty only if the controlled CPU study reveals a specific, reproducible inefficiency that is not adequately handled by the compared methods, followed by a new mechanism that addresses it.

This framing keeps the thesis feasible even if the first heuristic idea changes. The strongest result may be an algorithmic improvement, a carefully characterized failure regime, or both.

## 8. Six-Month Milestones

|             |                                                                   |
|-------------|-------------------------------------------------------------------|
| Weeks 1-4   | Codebase setup, baseline validation, ground-truth pipeline        |
| Weeks 5-8   | Selectivity/correlation sweep and reproducible failure maps       |
| Weeks 9-12  | CPU profiling, bottleneck decomposition, research-question freeze |
| Weeks 13-18 | Targeted heuristic design, C++ implementation, ablations          |
| Weeks 19-21 | Full benchmark, statistical checks, robustness tests              |
| Weeks 22-24 | Manuscript-ready draft, figures, reproducibility package          |

## 9. Selected References

1. Malkov, Y. A., & Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. IEEE TPAMI.

2. Subramanya, S. V., et al. (2019). DiskANN: Fast accurate billion-point nearest neighbor search on a single node. NeurIPS 32.

3. Gollapudi, S., et al. (2023). Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. WWW 2023, 3406-3416. DOI: 10.1145/3543507.3583552.

4. Patel, L., Kraft, P., Guestrin, C., & Zaharia, M. (2024). ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. Proceedings of the ACM on Management of Data, 2(3), Article 120. DOI: 10.1145/3654923.

5. Sehgal, G., & Salihoğlu, S. (2025). NaviX: A Native Vector Index Design for Graph DBMSs With Robust Predicate-Agnostic Search Performance. PVLDB, 18(11), 4438-4450. DOI: 10.14778/3749646.3749704.

6. Shi, J., Cai, Y., & Zheng, W. (2025). Filtered Approximate Nearest Neighbor Search: A Unified Benchmark and Systematic Experimental Study. arXiv:2509.07789.

7. Xu, H., et al. (2026). JAG: Joint Attribute Graphs for Filtered Nearest Neighbor Search. arXiv:2602.10258.

8. Kim, Y., & Choe, G. (2026). RACORN-1: Adaptive Recall-Preserving Speedup for Low-Selectivity Filtered Vector Search. arXiv:2607.00768.

9. Tumpa, F. A., & Gupta, R. (2026). BOA: Beamwidth Online Adaptation for Filtered-ANNS on a GPU. arXiv:2609.16175.

Prepared for supervisor discussion. Literature status checked against publicly available sources through September 20, 2026.